LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 3 TENTANG STACK

Disusun Oleh :

M.FAJAR FADHILUL ZIKRI

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, TAHUN 2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya, sehingga laporan praktikum ini dapat diselesaikan tepat pada waktunya. Laporan ini disusun sebagai dokumentasi dan evaluasi dari materi mata kuliah Struktur Data, yang berfokus pada cara mengorganisasi serta mengelola data secara logis dalam sistem komputer.

Praktikum ini memberikan pemahaman mengenai pentingnya pemilihan metode penyimpanan data yang tepat. Struktur data bukan sekadar cara menyimpan informasi, melainkan fondasi dalam membangun algoritma yang efisien. Dengan pengorganisasian data yang baik, sebuah program dapat memproses informasi dengan kecepatan yang lebih tinggi dan penggunaan memori yang lebih optimal.

Melalui rangkaian praktikum ini, mahasiswa diharapkan tidak hanya menguasai teori pengolahan data, tetapi juga mampu mengimplementasikannya dalam bentuk kode program yang sistematis. Kemampuan untuk menyusun data secara terstruktur merupakan keterampilan krusial yang dibutuhkan untuk memecahkan masalah komputasi yang lebih kompleks di masa mendatang.

Terima kasih kami sampaikan kepada dosen pembimbing atas bimbingan dan ilmu yang diberikan selama proses pembelajaran. Terima kasih juga kepada rekan-rekan mahasiswa yang telah memberikan dukungan serta berbagi ide selama praktikum berlangsung. Penyusun menyadari bahwa laporan ini masih memiliki ruang untuk perbaikan, sehingga kritik dan saran yang membangun akan sangat dihargai.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan membantu dalam memahami betapa pentingnya peran struktur data dalam dunia pengembangan perangkat lunak.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR……ii

DAFTAR ISI……..iii

BAB 1 PENDAHULUAN……..……1

1.1 Pengertian Praktikum……1

1.2 Tujuan Praktikum……..1

1.3 Persyaratan Praktikum…….……..1

1.4 Tempat dan Waktu Praktikum…….……..2

BAB 2 PEMBAHASAN PRAKTIKUM…….3

2.1 Pengertian Stack…….3

2.2 Langkah-Langkah Praktikum…….3

BAB 3 PENUTUP……..……15

3.1 Kesimpulan……...……15

LAMPIRAN……...……16

BAB 1
PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan cara pengorganisasian, penyimpanan, dan pengelolaan data secara efisien.

Kegiatan ini tidak hanya menekankan pada penguasaan teori mengenai model-model data, tetapi juga pada latihan implementasi struktur penyimpanan yang optimal, analisis efisiensi algoritma, hingga pengujian performa program. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual mengenai tata kelola data dengan kemampuan teknis dalam memecahkan masalah komputasi yang kompleks secara sistematis.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar struktur data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Struktur data.
3. Meningkatkan keterampilan dalam memecahkan masalah (problem solving) dengan pendekatan algoritmik.
4. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
5. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
6. Mengetahui dan mengaplikasikan Tipe Data stack, queue dll.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.

2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Environment (IDE) yang direkomendasikan.

3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.

4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.

5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum struktur data mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian satck

1. DEFINISI UMUM
Stack (tumpukan) adalah struktur data linear yang mengikuti prinsip LIFO (Last-In, First-Out). Artinya, elemen yang terakhir dimasukkan akan menjadi elemen yang pertama kali dikeluarkan.
2. ANALOGI
Bisa dibayangkan seperti tumpukan buku di dalam sebuah kotak sempit atau tumpukan piring. Untuk mengambil buku yang paling bawah, Anda harus mengambil semua buku yang ada di atasnya satu per satu.
3. ISTILAH PENTING
- TOP: Ujung atau posisi paling atas dari tumpukan (satu-satunya akses untuk menambah atau menghapus data).
- MAX: Batas maksimal kapasitas tumpukan.

4. OPERASI DASAR
- Push: Operasi untuk menambahkan atau memasukkan elemen baru ke bagian atas tumpukan.
- Pop: Operasi untuk menghapus atau mengeluarkan elemen dari posisi paling atas.
- Peek (Top): Operasi untuk melihat nilai elemen teratas tanpa menghapusnya.
- isEmpty: Mengecek apakah tumpukan dalam keadaan kosong.
- isFull: Mengecek apakah tumpukan sudah penuh.

5. KONDISI ERROR
- Stack Overflow: Kondisi error saat mencoba melakukan *Push* ke tumpukan yang sudah penuh.
- Stack Underflow: Kondisi error saat mencoba melakukan *Pop* atau *Peek* pada tumpukan yang kosong.

6. IMPLEMENTASI
Stack biasanya dibangun menggunakan dua cara:
- Array: Ukurannya statis (tetap sejak awal).
- Linked List: Ukurannya dinamis (bisa bertambah sesuai kebutuhan memori).

7. CONTOH PENERAPAN
- Mekanisme Undo/Redo: Menyimpan urutan aksi terakhir di aplikasi.
- Browser History: Menyimpan daftar situs yang dikunjungi untuk fungsi tombol "Back".
- Manajemen Memori: Digunakan dalam pemanggilan fungsi (*Call Stack*) pada sistem operasi.
- Parsing: Digunakan untuk mengecek keseimbangan tanda kurung pada kode program.

2.7 Langkah Langkah Praktikum

Pada praktikum kali ini kita akan membuat 5 buah class yang pertama Adalah **StackArray:**

```
public class StackArray_2511533023 {
    static final int MAX = 1000;
    int top;
    int a[] = new int[MAX];

    // Method untuk mengecek apakah stack sedang kosong
    boolean isEmpty_3023() {
        return (top < 0);
    }

    // Constructor untuk inisialisasi awal index top
    StackArray_2511533023() {
        top = -1;
    }

    // Method untuk memasukkan data ke dalam stack
    boolean push_3023(int x) {
        if (top >= (MAX - 1)) {
            System.out.println("Stack Overflow");
            return false;
        } else {
            a[++top] = x;
            System.out.println(x + " dimasukkan dalam stack");
            return true;
        }
    }

    // Method untuk mengambil dan menghapus data teratas
    int pop_3023() {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        } else {
            int x = a[top--];
            return x;
        }
    }

    // Method untuk melihat data teratas tanpa menghapusnya
    int peek_3023() {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
```

```
        } else {
            int x = a[top];
            return x;
        }
    }

    // Method untuk mencetak seluruh isi stack ke layar
    void print_3023() {
        for (int i = top; i > -1; i--) {
            System.out.print(" " + a[i]);
        }
    }
}
```

Hasil Output:
Tidak ada karena ini merupakan class blueprint atau kerangka saja yang nanti hasil nya dapat di panggil pakai class driver.

Class ke-2 adalah StackArrayDriver:

```
public class StackArrayDriver_2511533023 {
        public static void main (String [] args){
                StackArray_2511533023 s = new StackArray_2511533023();
                s.push_3023(10);
                s.push_3023(20);
                s.push_3023(30);
                System.out.println(s.pop_3023() + " dikeluarkan dari Stack");
                System.out.println("elemen teratas adalah :" + s.peek_3023());
                System.out.println("elemen pada stack :");
                s.print_3023();
        }

}
```

Hasil Output:
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari Stack
elemen teratas adalah :20
elemen pada stack :
20 10

Class ke-3 SiswaStack:

```
import java.util.ArrayList;

//Class untuk menyimpan data Siswa
class Siswa_2511533023 {
 String nama;
 int nim;
 public Siswa_2511533023(String nama, int nim) {
    this.nama = nama;
    this.nim = nim;
 }

 public String toString_3023() {
    return "Nim: " + nim + ", Nama: " + nama;
 }
}

//Class Utama untuk Struktur Data Stack
public class SiswaStack_2511533023 {
 // Tipe data di dalam ArrayList harus Siswa_2511533023
 private ArrayList<Siswa_2511533023> stack;

 public SiswaStack_2511533023() {
    stack = new ArrayList<>();
 }

 public void push_3023(Siswa_2511533023 mhs) {
    stack.add(mhs);
 }

 public Siswa_2511533023 pop_3023() {
    if (!isEmpty_3023()) {
       return stack.remove(stack.size() - 1);
    }
    return null;
 }

 public Siswa_2511533023 peek_3023() {
    if (!isEmpty_3023()) {
       return stack.get(stack.size() - 1);
    }
    return null;
 }

 public boolean isEmpty_3023() {
    return stack.isEmpty();
```

```
    }

    public void tampilkanSiswa_3023() {
        for (int i = stack.size() - 1; i >= 0; i--) {
            System.out.println(stack.get(i).toString_3023());
        }
    }

    public static void main(String[] args) {
        SiswaStack_2511533023 studentStack = new SiswaStack_2511533023();

        // Membuat objek dari class Siswa_2511533023, bukan SiswaStack
        Siswa_2511533023 mhs1 = new Siswa_2511533023("Ali", 1);
        Siswa_2511533023 mhs2 = new Siswa_2511533023("Boby", 2);
        Siswa_2511533023 mhs3 = new Siswa_2511533023("Charles", 3);

        studentStack.push_3023(mhs1);
        studentStack.push_3023(mhs2);
        studentStack.push_3023(mhs3);

        System.out.println("Siswa di dalam stack:");
        studentStack.tampilkanSiswa_3023();

        System.out.println("\nsiswa teratas : " +
studentStack.peek_3023().toString_3023());

        Siswa_2511533023 popped = studentStack.pop_3023();
        System.out.println("mengeluarkan siswa teratas dari stack : " +
popped.toString_3023());

        System.out.println("\ndaftar siswa setelah di pop :");
        studentStack.tampilkanSiswa_3023();
    }
}
```

Hasil Output:

```
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali

siswa teratas : Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack : Nim: 3, Nama: Charles

daftar siswa setelah di pop :
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

Class ke-4 Mahasiswa:

```
import java.util.Stack;

public class NilaiMaksimum_2511533023 {
   // Method untuk mencari nilai maksimum di dalam stack tanpa menghilangkan data asli
   public static int max_3023(Stack<Integer> s) {
      // Stack pembantu untuk menampung data sementara agar data asli bisa dikembalikan
      Stack<Integer> backup = new Stack<Integer>();

      // Mengambil elemen pertama dari stack sebagai nilai maksimum awal
      int maxValue = s.pop();
      backup.push(maxValue); // Memasukkan ke backup agar data tidak hilang

      // Perulangan untuk memeriksa seluruh sisa elemen di dalam stack
      while (!s.isEmpty()) {
         int next = s.pop(); // Mengambil data berikutnya
         backup.push(next); // Simpan ke backup
         maxValue = Math.max(maxValue, next); // Bandingkan dan simpan nilai yang lebih besar
      }

      // Perulangan untuk mengembalikan semua data dari backup ke stack asli (s)
      while (!backup.isEmpty()) {
         s.push(backup.pop());
      }

      // Mengembalikan hasil nilai tertinggi yang ditemukan
      return maxValue;
   }

   public static void main(String[] args) {
      // Instansiasi objek stack dari library Java
      Stack<Integer> s = new Stack<Integer>();

      // Memasukkan data ke dalam stack
      s.push(70);
      s.push(12);
      s.push(20);

      // Menampilkan isi stack dan informasi elemen teratas
      System.out.println("isi stack " + s);
      System.out.println("Stack Teratas " + s.peek());

      // Memanggil method max_3023 dan menampilkan hasilnya
```

```
        System.out.println("Nilai maksimum " + max_3023(s));
    }
}
```

Hasil Output:
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali

siswa teratas : Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack : Nim: 3, Nama: Charles

daftar siswa setelah di pop :
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
.

Class ke-5 StackPostfix:

```
import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511533023 {
    // Method untuk mengevaluasi ekspresi matematika dalam bentuk Postfix
    public static int postfixEvaluate_3023(String expression) {
        // Stack untuk menyimpan angka (operan) sementara
        Stack<Integer> s = new Stack<Integer>();

        // Menggunakan Scanner untuk membaca setiap bagian dari string ekspresi
        try (Scanner input = new Scanner(expression)) {
            while (input.hasNext()) {
                // Jika bagian yang dibaca adalah angka (operan)
                if (input.hasNextInt()) {
                    s.push(input.nextInt()); // Masukkan angka ke dalam stack
                } else {
                    // Jika yang dibaca adalah operator (+, -, *, /)
                    String operator = input.next();

                    // Mengambil dua angka terakhir dari stack
                    // Angka terakhir yang masuk menjadi operan kedua
                    int operand2 = s.pop();
                    // Angka sebelumnya menjadi operan pertama
                    int operand1 = s.pop();

                    // Melakukan kalkulasi berdasarkan jenis operator
                    if (operator.equals("+")) {
                        s.push(operand1 + operand2);
```

```
                } else if (operator.equals("-")) {
                    s.push(operand1 - operand2);
                } else if (operator.equals("*")) {
                    s.push(operand1 * operand2);
                } else {
                    // Pembagian
                    s.push(operand1 / operand2);
                }
            }
        }
    }

    // Mengembalikan hasil akhir yang tersisa di dalam stack
    return s.pop();
  }

  public static void main(String[] args) {
    // Mencetak hasil evaluasi ekspresi "5 2 4 * + 7 -"
    System.out.println("hasil postfix = " + postfixEvaluate_3023("5 2 4 * + 7 -"));
  }
}
```

Hasil Output:

hasil postfix = 6

tujuan dari program ini Adalah sebagai **Postfix Evaluator** (Pengevaluasi Notasi Postfix). cara penulisan ekspresi matematika di mana operator diletakkan **setelah** angka-angkanya.

BAB 3
PENUTUP

3.1 Kesimpulan

dari praktikum ini adalah saya jadi paham kalau **Stack** itu bukan sekadar teori, tapi struktur data yang sangat berguna untuk mengatur urutan penyimpanan data dengan prinsip **LIFO (Last In First Out)**, di mana data yang paling terakhir dimasukkan justru yang akan pertama kali keluar. Melalui latihan tadi, saya belajar bahwa Stack bisa diterapkan untuk banyak hal, mulai dari yang sederhana seperti menyimpan data siswa, mencari nilai tertinggi dalam tumpukan, sampai membantu komputer menghitung ekspresi matematika kompleks seperti notasi postfix. Intinya, Stack mempermudah pengelolaan data yang butuh sistem "tumpukan" secara rapi dan sistematis agar proses pengambilan datanya jadi lebih cepat dan tidak berantakan.

Tugas Stack:
Class pertama Website_2511533023:

```
public class Website_2511533023 {
    private String judul_3023;
    private String url_3023;

    // Constructor untuk menginisialisasi atribut
    public Website_2511533023(String judul, String url) {
        this.judul_3023 = judul;
        this.url_3023 = url;
    }

    // Selektor (Getter)
    public String getJudul_3023() {
        return judul_3023;
    }

    public String getUrl_3023() {
        return url_3023;
    }

    // Mutator (Setter)
    public void setJudul_3023(String judul) {
        this.judul_3023 = judul;
    }

    public void setUrl_3023(String url) {
        this.url_3023 = url;
    }
}
```

Class ini berfungsi sebagai kerangka / blueprint yang akan dikendalikan oleh class browser.

Class kedua Browser_2511533023:

```
import java.util.Stack;
import java.util.Scanner;

public class Browser_2511533023 {
    public static void main(String[] args) {
        // Menggunakan java.util.Stack dengan tipe objek Website_2511533023
        Stack<Website_2511533023> riwayatStack = new Stack<>();
        Scanner input = new Scanner(System.in);
        int pilihan;

        do {
            // Tampilan Menu sesuai contoh output
```

```
System.out.println("\n= Browser History NIM: 2511533023 =");
System.out.println("1. Kunjungi Website (Push)");
System.out.println("2. Tombol Back (Pop)");
System.out.println("3. Lihat Halaman Aktif (Peek)");
System.out.println("4. Keluar");
System.out.print("Pilihan: ");
pilihan = input.nextInt();
input.nextLine(); // Clear buffer

switch (pilihan) {
    case 1:
        // Logika Push: Menambah data ke atas tumpukan
        System.out.print("Masukkan Judul: ");
        String judul = input.nextLine();
        System.out.print("Masukkan URL: ");
        String url = input.nextLine();

        riwayatStack.push(new Website_2511533023(judul, url));
        System.out.println("Berhasil mengunjungi halaman!");
        break;

    case 2:
        // Logika Pop: Menghapus halaman teratas
        // Penanganan agar tidak error saat stack kosong
        if (!riwayatStack.isEmpty()) {
            Website_2511533023 dihapus = riwayatStack.pop();
            System.out.println("Tombol Back ditekan. Menutup: " +
dihapus.getJudul_3023());
        } else {
            System.out.println("Riwayat kosong! Tidak ada halaman untuk
kembali.");
        }
        break;

    case 3:
        // Logika Peek: Melihat halaman tanpa menghapus
        if (!riwayatStack.isEmpty()) {
            Website_2511533023 aktif = riwayatStack.peek();
            System.out.println("Halaman Aktif: " + aktif.getJudul_3023());
            System.out.println("URL: " + aktif.getUrl_3023());
        } else {
            System.out.println("Tidak ada halaman yang sedang aktif.");
        }
        // Cek Status History (Jumlah total)
        System.out.println("Total riwayat tersimpan: " + riwayatStack.size());
        break;
```

```
            case 4:
                System.out.println("Keluar dari Browser...");
                break;

            default:
                System.out.println("Pilihan tidak valid!");
        }
    } while (pilihan != 4);

    input.close();
  }
}
```

Hasil Output:

```
= Browser History NIM: 2511533023 =
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

= Browser History NIM: 2511533023 =
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 3
Halaman Aktif: SSO UNAND
URL: https://sso.unand.ac.id/login
Total riwayat tersimpan: 1

= Browser History NIM: 2511533023 =
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 2
Tombol Back ditekan. Menutup: SSO UNAND
```

= Browser History NIM: 2511533023 =
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 4
Keluar dari Browser...

Fungsi dari program ini Adalah memanggil atribut yang sudah kita bikin di class website yang di mana itu merupakan kerangka /blueprint untuk program ini.